

NominaFlow

Informe Ejecutivo

Stack tecnologico, guardrails y roadmap de mejora

Fecha: 17 de July de 2026

Este documento resume las decisiones tecnicas, los mecanismos de control sobre la inteligencia artificial y las lineas de mejora priorizadas para el proyecto NominaFlow.

Indice

1. Resumen ejecutivo
2. Stack tecnologico y justificacion
3. Guardrails de inteligencia artificial
4. Mejoras futuras
5. Conclusion

Este informe esta redactado para lectores tecnicos y no tecnicos. Se ha evitado la jerga innecesaria y se han separado los conceptos para que nada se solape.

1. Resumen ejecutivo

NominaFlow es una aplicacion web que permite extraer datos estructurados de nominas espanolas a partir de archivos PDF o imagen. El sistema combina vision por computador, modelos de lenguaje de gran tamaño y reglas de validacion de negocio para convertir documentos no estructurados en datos verificables.

El proyecto no asume que la inteligencia artificial acierta siempre. Por el contrario, esta disenado para detectar errores, dudas e inconsistencias y marcarlos para revision humana. Esa postura es el eje central de la arquitectura y de la estrategia de producto.

Puntos clave

- Backend en Python con FastAPI, Pydantic y SQLAlchemy sobre SQLite.
- Frontend en Next.js 16 con React 19, TypeScript y Tailwind CSS.
- Extraccion mediante GPT-4o a traves de GitHub Models.
- Validacion de IBAN, fechas, ingresos, nombres y clasificacion del documento.
- Base de datos SQLite para persistencia simple y portatil.

Estado actual: producto funcional en desarrollo activo. Faltan tests automatizados, logging estructurado y algunos guardrails adicionales que se detallan en las secciones siguientes.

2. Stack tecnologico y justificacion

A continuacion se describe cada tecnologia principal, agrupada por capa, junto con la razon por la que fue seleccionada. Las decisiones buscan minimizar la complejidad operativa sin sacrificar calidad de codigo ni capacidad de crecimiento.

2.1. Backend

Python 3.10+

Lenguaje maduro, con gran ecosistema en ciencia de datos, procesamiento de documentos e integracion con APIs de inteligencia artificial. Su sintaxis moderna permite tipado progresivo y uniones con el operador |.

FastAPI

Framework web asincrono que genera documentacion OpenAPI automaticamente, valida requests con Pydantic y reduce la cantidad de codigo repetitivo. Es mas rapido de desarrollar y mas robusto que Flask para APIs JSON tipadas.

Pydantic 2

Define los contratos de la API y valida tipos en runtime. Garantiza que los datos que entran y salen del backend cumplen una estructura conocida, lo que reduce errores silenciosos.

SQLAlchemy 2.0

ORM estandar de Python. Abstrae las operaciones de base de datos y permite cambiar a PostgreSQL en el futuro sin reescribir la logica de negocio.

SQLite

Base de datos embebida. Elimina la necesidad de levantar un servidor de base de datos para el challenge y es suficiente para el volumen de datos esperado en un MVP.

PyMuPDF

Convierte paginas de PDF a imagenes con alta calidad. Es necesario porque el modelo de vision recibe imagenes, no documentos binarios.

OpenAI SDK / GitHub Models

Cliente oficial para llamar a GPT-4o. GitHub Models ofrece una puerta de acceso compatible con OpenAI, simplificando la autentificacion y el cambio de proveedor si fuera necesario.

2.2. Frontend

Next.js 16

Proporciona App Router, rendering hibrido, optimizacion de fuentes e imagenes, y un proxy reverso util para conectar frontend y backend durante el desarrollo sin problemas de CORS.

React 19

Biblioteca de componentes sobre la que se construye Next.js. La version 19 aporta mejoras en hidratacion y manejo de formularios.

TypeScript 5

Tipado estatico que detecta errores antes de ejecutar el codigo. Especialmente valioso en un frontend que consume una API externa.

Tailwind CSS 4

Framework de utilidades CSS que acelera el diseno consistente y reduce la cantidad de CSS custom que hay que mantener.

react-dropzone

Componente maduro y accesible para arrastrar y soltar archivos. Ahorra tiempo de desarrollo y mejora la experiencia de usuario.

2.3. Infraestructura y tooling

- uv: gestor de paquetes y entornos virtuales de Python. Es mas rapido que pip y genera lockfiles reproducibles.
- npm workspaces + Turbo: organizan el frontend como monorepo y aceleran los builds mediante cache.
- Prettier y ESLint: mantienen el formato y la calidad estatica del codigo.

3. Guardrails de inteligencia artificial

Un guardrail es un mecanismo de control que limita el comportamiento de un sistema de inteligencia artificial para que sus resultados sean mas seguros, fiables y alineados con las reglas de negocio. En NominaFlow, los guardrails actuan antes, durante y despues de la extraccion.

3.1. Guardrails implementados

Prompt engineering

El prompt al modelo incluye instrucciones estrictas de formato JSON, reglas para cada campo y una peticion explicita de confianza. Reduce las respuestas creativas o fuera de formato.

Parametros del modelo

Se usa temperature=0.1 para minimizar la creatividad y max_tokens=1024 para acotar la longitud de la respuesta.

Clasificacion del documento

El modelo devuelve el campo es_nomina antes de extraer datos. Si el documento no es una nomina, el sistema marca el resultado como needs_review y omite validaciones secundarias irrelevantes.

Validacion de formato

- IBAN: 24 caracteres, prefijo ES y digitos validos tras el prefijo.
- Fecha: formatos MM/YYYY o DD/MM/YYYY, no futura, no anterior a 2020.
- Nombres: al menos dos palabras y sin secuencias numericas sospechosas.
- Empresa: no vacia, mas de dos caracteres y no solo numeros.

Validacion de coherencia

- Ingresos netos menores que ingresos brutos.
- Alerta si la deduccion es menor al 5% o mayor al 50%.
- Deteccion de netos iguales a brutos.

Niveles de confianza y estados

Los resultados se clasifican en high, medium o low y se asigna un status validated o needs_review. Esto permite al usuario saber de un vistazo si debe revisar la extraccion.

Auditoria

Cada respuesta cruda del modelo se almacena en raw_llm_response. Permite investigar despues por que un campo fue extraido de una determinada manera.

3.2. Limitaciones actuales

- El modelo no esta obligado a devolver JSON valido mediante response_format. El parsing sigue siendo fragil.
- No se valida el checksum completo del IBAN espanol.
- No hay rate limiting ni control de abuso en los endpoints.
- No hay logging estructurado ni trazabilidad de peticiones.
- No se detectan intentos de prompt injection ni documentos maliciosos.
- No hay reintentos ni modelo alternativo ante fallos del LLM.

4. Mejoras futuras

A continuacion se presentan las mejoras organizadas por horizonte temporal y por impacto esperado en seguridad, fiabilidad y mantenibilidad.

4.1. Corto plazo

1. Activar JSON mode en la llamada a OpenAI para forzar respuestas JSON validas y reducir errores de parsing.
2. Implementar validacion de checksum del IBAN espanol con el algoritmo MOD-97-10.
3. Anadir rate limiting por IP o por usuario para proteger los endpoints de upload y extraccion.
4. Reemplazar los print por logging estructurado con identificadores de trazabilidad.
5. Crear tests unitarios para el validador y tests de integracion para el endpoint de subida.

4.2. Medio plazo

1. Construir una capa de API centralizada en el frontend con manejo de errores, timeouts y cancelacion de peticiones.
2. Validar las respuestas de la API en el frontend con Zod para detectar cambios de contrato en runtime.
3. Implementar reintentos con backoff exponencial ante fallos del modelo de lenguaje.
4. Anadir un segundo modelo de fallback para casos en los que GPT-4o falle o no este disponible.
5. Separar el servicio OCR en componentes mas pequenos: conversor de archivos, constructor de prompts, cliente LLM y parser de respuestas.

4.3. Largo plazo

1. Migrar de SQLite a PostgreSQL si el volumen de usuarios o datos lo justifica.
2. Anadir autentificacion y autorizacion para multiusuario.
3. Construir un dashboard de calidad con metricas de extraccion, tasas de error y revision humana.
4. Implementar deteccion de documentos maliciosos y sandboxing de archivos antes de procesarlos.
5. Evaluar modelos open source o locales para reducir costes y latencia en alta carga.

5. Conclusion

NominaFlow es un producto funcional que demuestra como construir un sistema de extraccion inteligente sin confiar ciegamente en la inteligencia artificial. La combinacion de FastAPI, Pydantic, SQLAlchemy, GPT-4o y Next.js proporciona una base solida, mientras que el motor de validacion y el campo es_nomina aportan el primer nivel de control.

Las proximas iteraciones deben centrarse en robustecer los guardrails: JSON mode, checksum de IBAN, rate limiting, logging estructurado y tests automatizados. Estas mejoras elevan el sistema de un MVP demostrable a una solucion preparada para entornos con usuarios reales.

Recomendacion prioritaria: implementar JSON mode del LLM y la validacion de checksum del IBAN como siguiente paso. Son cambios de bajo riesgo y alto impacto en la fiabilidad del producto.